PRAKTIKUM 5
KONTROL LED PADA NODEMCU ESP32 MENGGUNAKAN APLIKASI TELEGRAM

NRP	: 1. 3223600046
	2. 3223600047
	3. 3223600056
	4. 3223600058
	5. 3223600060
Nama	: 1. Hari Tammim Mu'izha
	2. Bintang Endra Erlangga
	3. Mochamad Najib Thariq
	4. Ihsanta Zaki Sanjaya
	5. Mochammad Ega Yudhistira
Kelas	: 2 Teknik Komputer B
Mata Kuliah	: Praktikum Alat Pengembangan Perangkat Lunak
Tanggal	: Kamis, 20 Maret 2025
Dosen	: Dr. Setiawardhana, S.T., M.T.

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

KONTROL LED PADA NODEMCU ESP32 MENGGUNAKAN APLIKASI TELEGRAM

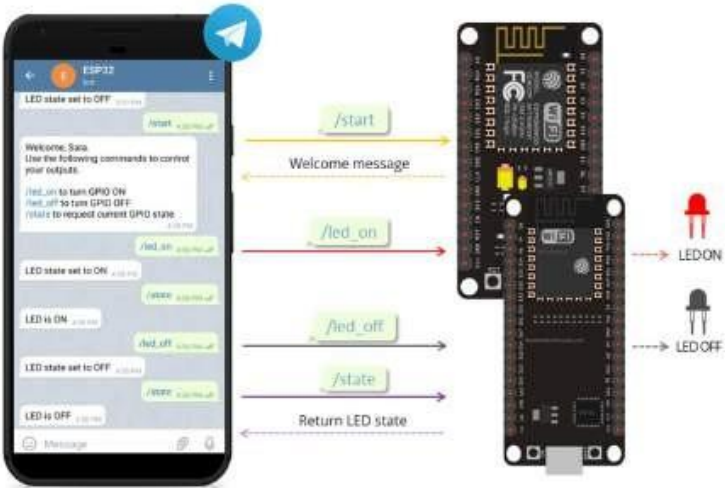
A. DASAR TEORI

DESCRIPTION

Pada pembahasan kali ini kita akan mencoba men-kontrol ESP32 atau ESP8266 darimana saja menggunakan aplikasi Telegram. Pada contoh ini masih sama yaitu menggunakan Led sebagai media untuk dikontrol, selain Led anda bebas men-kontrol apa saja seperti motor servo atau motor dc. Untuk mengkontrol led atau aktuator tersebut, kita hanya tinggal mengirim pesan melalui chat di telegram. Pemrograman ESP menggunakan Arduino IDE.

Gambar 1. Arsitektur kontrol Led menggunakan Telegram Chat

PROJECT OVERVIEW



Gambar 2. Cara kirim pesan ke board NodeMCU

Keterangan IoT dengan Telegram bot pada gambar adalah :

1. Anda akan membuat bot Telegram untuk ESP32 / ESP8266 Anda;
2. Anda dapat memulai percakapan dengan bot;
3. Saat Anda mengirim pesan /led_on ke bot, ESP menerima pesan tersebut dan mengaktifkan GPIO 2;
4. Demikian pula, saat Anda mengirim pesan /led_off, GPIO 2 akan mati;
5. Selain itu, Anda juga dapat mengirim pesan /state untuk meminta status GPIO saat ini. Saat ESP menerima pesan itu, bot merespons dengan status GPIO saat ini;
6. Anda dapat mengirim /start pesan untuk menerima pesan selamat datang dengan perintah untuk mengontrol board NodeMCU.

INTRODUCTION TELEGRAM

Telegram Messenger adalah layanan olah pesan cepat dan suara melalui IP berbasis cloud. Anda dapat dengan mudah menginstalnya di smartphone (Android dan iPhone) atau komputer (PC, Mac, dan Linux). Ini gratis dan Telegram memungkinkan Anda membuat bot (robochat) yang dapat Anda gunakan untuk berinteraksi.

"Bot adalah aplikasi pihak ketiga yang berjalan di dalam Telegram. Pengguna dapat berinteraksi dengan bot dengan mengirimi mereka pesan, perintah, dan permintaan sebaris. Anda mengontrol bot Anda menggunakan permintaan HTTPS ke Telegram Bot API ".

ESP32 / ESP8266 akan berinteraksi dengan bot Telegram untuk menerima dan menangani pesan, kemudian memberikan tanggapan. Dalam tutorial ini Anda akan mempelajari cara menggunakan Telegram untuk mengirim pesan ke bot Anda untuk mengontrol keluaran ESP dari mana saja (Anda hanya perlu Telegram dan akses ke internet).

B. ALAT DAN BAHAN

1. ESP32 Board
2. Laptop/PC
3. Arduino IDE
4. USB Micro
5. Telegram

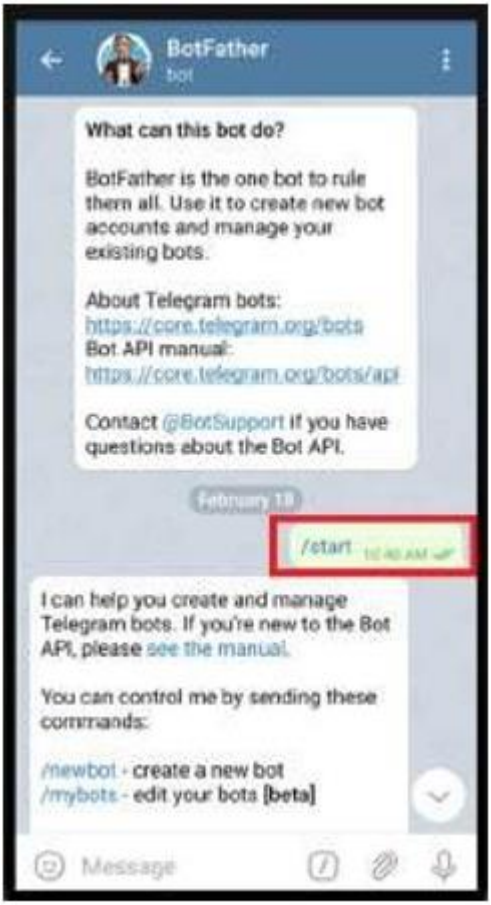
C. PROSEDUR PERCOBAAN

CREATING TELEGRAM BOT

1. Download Aplikasi Telegram dan Install

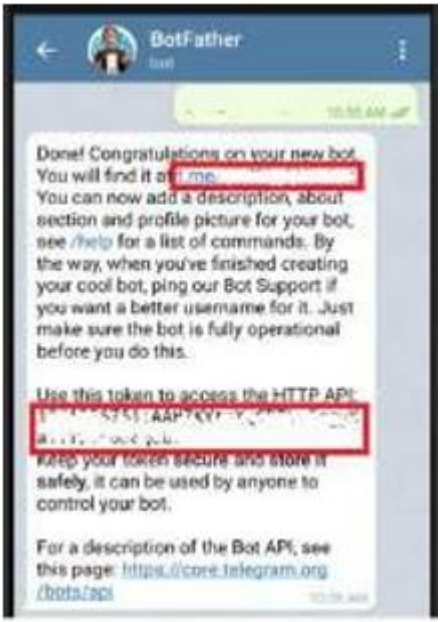
2. Cari "botfather" dan klik BotFather seperti yang ditunjukkan pada gambar di bawah ini. Atau buka tautan ini ***t.me/botfather*** di ponsel Anda.

3. Jendela berikut akan terbuka dan Anda akan diminta untuk mengklik tombol **start**.



4. Ketik **/newbot** dan ikuti petunjuk untuk membuat bot Anda. Buat **name** dan **username.**

5. Jika bot Anda berhasil dibuat, Anda akan menerima pesan dengan tautan untuk mengakses bot dan token bot. Simpan token bot karena Anda akan membutuhkannya agar ESP32 / ESP8266 dapat berinteraksi dengan bot.



GET YOUR TELEGRAM USER ID

Siapa pun yang mengetahui nama pengguna Telegram bot Anda maka dapat berinteraksi dengannya. Untuk memastikan agar mengabaikan pesan yang bukan dari akun Telegram kami (atau pengguna resmi), Anda bisa mendapatkan ID Pengguna Telegram Anda. Kemudian, ketika bot telegram Anda menerima pesan, ESP dapat memeriksa apakah ID pengirim sesuai dengan ID Pengguna Anda sehingga bisa menangani pesan atau mengabaikannya.

Di akun Telegram Anda, cari "IDBot" atau buka tautan ini t.me/myidbot di ponsel Anda.

Mulailah percakapan dengan bot itu dan ketik **/getid**. Anda akan mendapatkan balasan kembali dengan ID pengguna Anda. Simpan ID pengguna itu, karena Anda akan membutuhkannya nanti dalam praktikum ini.

PREPARING ARDUINO IDE

Install ESP32 atau ESP8266 anda di Arduino IDE

UNIVERSAL LIBRARY TELEGRAM BOT

Untuk berinteraksi dengan bot Telegram, kami akan menggunakan library dari Universal Bot Telegram yang dibuat oleh Brian Lough yang menyediakan antarmuka yang mudah untuk API Bot Telegram.

1. Unduh library **Universal-Arduino-Telegram-Bot-master.zip**
2. 2.Selanjutnya tambahkan .zip ke arduino IDE : ke **Sketch> Include Library> Add.ZIP Library ...**

ArduinoJson LIBRARY

Anda juga harus menginstal library ArduinoJson.

1. Buka Skech> Include Library> Manages Libraries
2. Cari “ArduinoJson” dan Instal.

HARDWARE REQUIRED

1. .ESP board
2. 2.Led GPIO 2

CONTROL OUTPUT USING TELEGRAM

Program ini kompatibel dengan board NodeMCU ESP32 dan ESP8266. Kode akan memuat library yang

tepat sesuai dengan board yang dipilih.

```
/*
 Rui Santos
 Complete project details at
https://RandomNerdTutorials.com/telegram-control-
esp32-esp8266-nodemcu-outputs/
 Project created using Brian Lough's Universal
Telegram Bot Library:
 https://github.com/witnessmenow/Universal-
Arduino-Telegram-Bot
 Example based on the Universal Arduino Telegram
Bot Library:
 https://github.com/witnessmenow/Universal-
Arduino-Telegram-
Bot/blob/master/examples/ESP8266/FlashLED/FlashLED
.ino
*/

#ifdef ESP32
#include <WiFi.h>
#else
#include <ESP8266WiFi.h>
#endif

#include <WiFiClientSecure.h>
#include  <UniversalTelegramBot.h>  //  Universal
Telegram Bot Library written by Brian Lough
#include <ArduinoJson.h>

// Replace with your network credentials
const char* ssid = "REPLACE_WITH_YOUR_SSID";
const char* password =
"REPLACE_WITH_YOUR_PASSWORD";

// Initialize Telegram BOT
#define BOTtoken
"XXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
// Your Bot Token (Get from Botfather)
#define CHAT_ID "XXXXXXXXXX"

#ifdef ESP8266
X509List cert(TELEGRAM_CERTIFICATE_ROOT);
#endif
```

```
WiFiClientSecure client;
UniversalTelegramBot bot(BOTtoken, client);

// Checks for new messages every 1 second.
int botRequestDelay = 1000;
unsigned long lastTimeBotRan;

const int ledPin = 2;
bool ledState = LOW;

// Handle new messages
void handleNewMessages(int numNewMessages) {
    Serial.println("handleNewMessages");
    Serial.println(String(numNewMessages));

    for (int i = 0; i < numNewMessages; i++) {
        // Chat ID of the requester
        String              chat_id              =
String(bot.messages[i].chat_id);
        if (chat_id != CHAT_ID) {
            bot.sendMessage(chat_id, "Unauthorized
user", "");
            continue;
        }

        // Print the received message
        String text = bot.messages[i].text;
        Serial.println(text);
        String             from_name             =
bot.messages[i].from_name;

        if (text == "/start") {
            String   welcome   =   "Welcome,   "   +
from_name + ".\n";
            welcome += "Use the following commands
to control your outputs.\n\n";
            welcome  +=  "/led_on  to  turn  GPIO  ON
\n";
            welcome += "/led_off to turn GPIO OFF
\n";
            welcome += "/state to request current
GPIO state \n";
            bot.sendMessage(chat_id, welcome, "");
        }

        if (text == "/led_on") {
```

```
            bot.sendMessage(chat_id,  "LED  state
set to ON", "");
            ledState = HIGH;
            digitalWrite(ledPin, ledState);
        }

        if (text == "/led_off") {
            bot.sendMessage(chat_id,  "LED  state
set to OFF", "");
            ledState = LOW;
            digitalWrite(ledPin, ledState);
        }

        if (text == "/state") {
            if (digitalRead(ledPin)) {
                bot.sendMessage(chat_id,  "LED  is
ON", "");
            } else {
                bot.sendMessage(chat_id,  "LED  is
OFF", "");
            }
        }
    }
}

void setup() {
    Serial.begin(115200);

    #ifdef ESP8266
    configTime(0, 0, "pool.ntp.org"); // Get UTC
time via NTP
    client.setTrustAnchors(&cert);  //  Add  root
certificate for api.telegram.org
    #endif

    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, ledState);

    // Connect to Wi-Fi
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    #ifdef ESP32
    client.setCACert(TELEGRAM_CERTIFICATE_ROOT);
// Add root certificate for api.telegram.org
    #endif
```

```
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.println("Connecting to WiFi..");
    }

    // Print ESP32 Local IP Address
    Serial.println(WiFi.localIP());
}

void loop() {
    if     (millis()     >     lastTimeBotRan     +
botRequestDelay) {
        int            numNewMessages            =
bot.getUpdates(bot.last_message_received + 1);
        while (numNewMessages) {
            Serial.println("got response");
            handleNewMessages(numNewMessages);
            numNewMessages                          =
bot.getUpdates(bot.last_message_received + 1);
        }
        lastTimeBotRan = millis();
    }
}
```

DEMONSTRATION

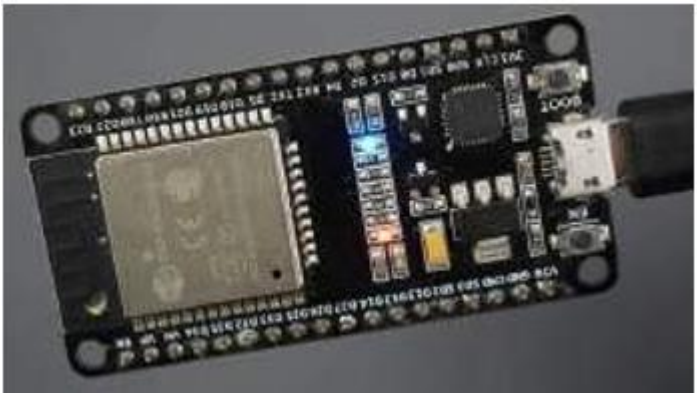
Upload program ke board ESP32 atau ESP8266 Anda. Tools> Board dan pilih board yang Anda gunakan. Pergi ke Tools> Port dan pilih port COM yang terhubung ke papan Anda.

Setelah mengupload program, tekan tombol EN/RST on-board ESP32/ESP8266 sehingga mulai menjalankan program. Kemudian, Anda dapat membuka Monitor Serial untuk memeriksa jalannya program.

Buka akun Telegram Anda dan buka chat dengan bot Anda. Kirim perintah berikut dan lihat bot merespons:

/start menampilkan pesan selamat datang dengan perintah yang valid.
/led_on menyalakan LED.
/led_off mematikan LED.
/state meminta status LED saat ini.

LED on-board akan menyala dan mati sesuai perintah (khsusus LED on-board ESP8266 bekerja secara terbalik, mati saat Anda mengirim **/led_on** dan menyala saat Anda mengirim **/led_off**).



Pada saat yang sama, serial monitor juga menampilkan respon berikut :

Jika Anda mencoba berinteraksi dengan bot dari akun lain, Anda akan mendapatkan pesan "Pengguna tidak resmi".

Dalam tutorial ini Anda telah mempelajari cara membuat Telegram Bot untuk berinteraksi dengan ESP32 atau ESP8266. Dengan bot ini, **Anda dapat menggunakan akun Telegram Anda untuk mengirim pesan ke ESP dan mengontrol keluarannya. ESP juga dapat berinteraksi dengan bot untuk mengirim tanggapan.** Selama Anda memiliki koneksi internet (dan board Anda juga), Anda dapat mengontrol dan memantau semua peralatan dari mana saja di dunia.

D. PERCOBAAN

1. Program

```
/*
 Rui Santos
```

```
 Complete project details at
https://RandomNerdTutorials.com/telegram-
control-esp32-esp8266-nodemcu-outputs/
 Project created using Brian Lough's Universal
Telegram Bot Library:
 https://github.com/witnessmenow/Universal-
Arduino-Telegram-Bot
 Example based on the Universal Arduino
Telegram Bot Library:
 https://github.com/witnessmenow/Universal-
Arduino-Telegram-
Bot/blob/master/examples/ESP8266/FlashLED/Flash
LED.ino
*/

#ifdef ESP32
#include <WiFi.h>
#else
#include <ESP8266WiFi.h>
#endif

#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>  // Universal
Telegram Bot Library by Brian Lough
#include <ArduinoJson.h>

// Replace with your network credentials
const char* ssid = "Foldexz";
const char* password = "12345678";
```

```
// Initialize Telegram BOT
#define BOTtoken
"8162309654:AAFYAaKQztC_G1vgaeLG4f3mKZllhV8NWoM
"
#define CHAT_ID "5851519974"  // Your Chat ID

#ifdef ESP8266
X509List cert(TELEGRAM_CERTIFICATE_ROOT);
#endif

WiFiClientSecure client;
UniversalTelegramBot bot(BOTtoken, client);

// Bot request delay (checks for new messages
every 1 second)
int botRequestDelay = 1000;
unsigned long lastTimeBotRan;

const int ledPin = 2;
bool ledState = LOW;

// Handle received messages
void handleNewMessages(int numNewMessages) {
    Serial.println("handleNewMessages");
    Serial.println(String(numNewMessages));

    for (int i = 0; i < numNewMessages; i++) {
        // Chat ID of the requester
        String chat_id =
String(bot.messages[i].chat_id);
```

```
        if (chat_id != CHAT_ID) {
            bot.sendMessage(chat_id,
"Unauthorized user", "");
            continue;
        }

        // Print the received message
        String text = bot.messages[i].text;
        Serial.println(text);

        String from_name =
bot.messages[i].from_name;

        if (text == "/start") {
            String welcome = "Welcome, " +
from_name + ".\n";
            welcome += "Use the following
commands to control your outputs.\n\n";
            welcome += "/led_on to turn GPIO ON
\n";
            welcome += "/led_off to turn GPIO
OFF \n";
            welcome += "/state to request
current GPIO state \n";
            bot.sendMessage(chat_id, welcome,
"");
        }

        if (text == "/led_on") {
            bot.sendMessage(chat_id, "LED state
set to ON", "");
```

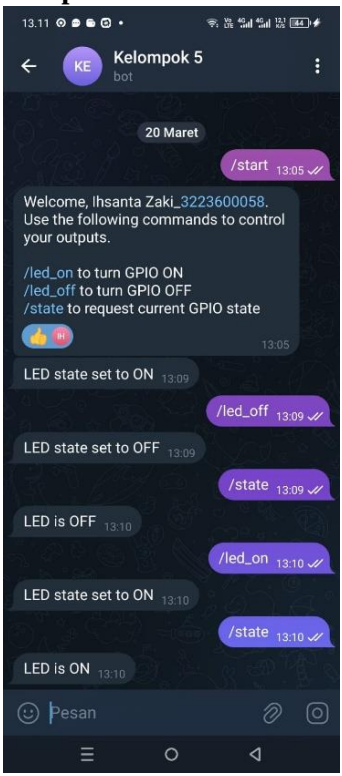
```
            ledState = HIGH;
            digitalWrite(ledPin, ledState);
        }

        if (text == "/led_off") {
            bot.sendMessage(chat_id, "LED state
set to OFF", "");
            ledState = LOW;
            digitalWrite(ledPin, ledState);
        }

        if (text == "/state") {
            if (digitalRead(ledPin)) {
                bot.sendMessage(chat_id, "LED
is ON", "");
            } else {
                bot.sendMessage(chat_id, "LED
is OFF", "");
            }
        }
    }
}

void setup() {
    Serial.begin(115200);

#ifdef ESP8266
    configTime(0, 0, "pool.ntp.org");  // Get
UTC time via NTP
    client.setTrustAnchors(&cert);     // Add
root certificate for api.telegram.org
```

```
#endif

    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, ledState);

    // Connect to Wi-Fi
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

#ifdef ESP32
    client.setCACert(TELEGRAM_CERTIFICATE_ROOT)
;  // Add root certificate for api.telegram.org
#endif

    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.println("Connecting to WiFi..");
    }

    // Print ESP32 Local IP Address
    Serial.println(WiFi.localIP());
}

void loop() {
    if (millis() > lastTimeBotRan +
botRequestDelay) {
        int numNewMessages =
bot.getUpdates(bot.last_message_received + 1);

        while (numNewMessages) {
            Serial.println("Got response");
```

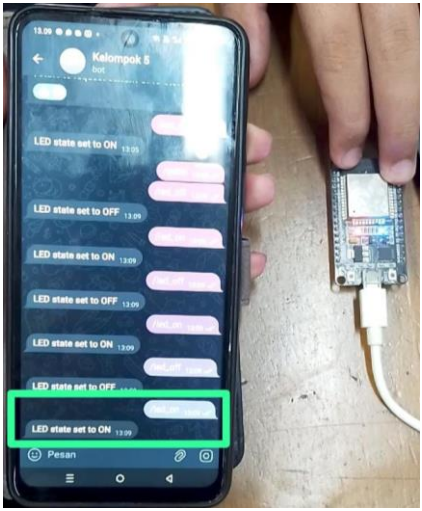
```
            handleNewMessages(numNewMessages);
            numNewMessages =
bot.getUpdates(bot.last_message_received + 1);
        }

        lastTimeBotRan = millis();
    }
}
```

2. **Hasil Program**

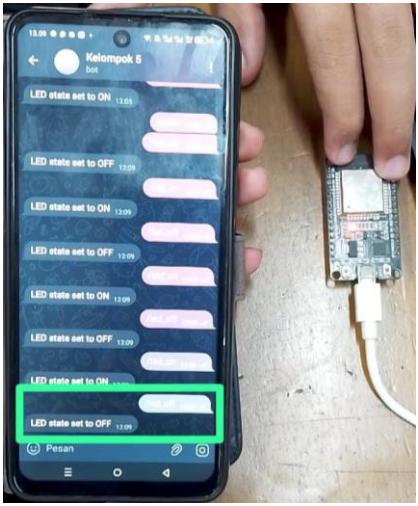
- **Tampilan Pesan Bot**



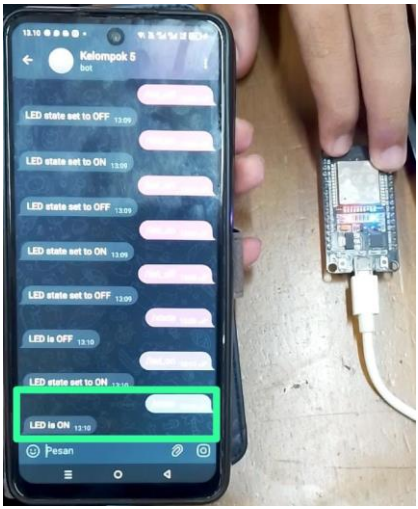
- **LED ON**



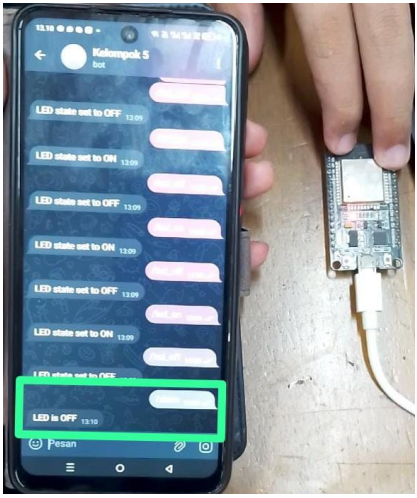
- **LED OFF**



- **STATE LED ON**



- **STATE LED OFF**



Hasil percobaan juga dapat dilihat melalui link video berikut:

https://drive.google.com/file/d/11FpmHWxpUPppzWGIQmMZEpQ_e6pwFEcl/view?usp=drivesdk

E. ANALISA

Pada praktikum ini, kami melakukan implementasi sistem kontrol LED menggunakan NodeMCU ESP32 yang dikendalikan melalui bot Telegram. Program diawali dengan mengimpor berbagai pustaka yang diperlukan, seperti WiFi, WiFiClientSecure, UniversalTelegramBot, dan ArduinoJson. Selanjutnya, kredensial jaringan WiFi dan token bot Telegram diinisialisasi agar ESP32 dapat terhubung ke internet dan berkomunikasi dengan bot.

Pada fungsi setup(), perangkat akan menghubungkan dirinya ke jaringan WiFi menggunakan WiFi.begin(ssid, password), dan menunggu hingga koneksi berhasil. Setelah itu, perangkat akan mencetak alamat IP lokalnya ke serial monitor. LED yang terhubung ke pin GPIO 2 diatur sebagai output, dan status awalnya diatur ke LOW (mati).

Pada fungsi loop(), perangkat secara berkala memeriksa apakah ada pesan baru yang diterima dari bot Telegram. Jika ada, pesan akan diproses oleh fungsi handleNewMessages(int numNewMessages). Setiap pesan yang masuk diperiksa apakah berasal dari CHAT_ID yang telah diotorisasi. Jika tidak, bot akan mengirimkan pesan "Unauthorized user". Jika perintah yang diterima adalah /start, bot akan mengirimkan pesan berisi daftar perintah yang dapat digunakan. Perintah /led_on akan mengubah status LED menjadi

HIGH (menyala) dan mengirimkan konfirmasi "LED state set to ON". Sebaliknya, perintah /led_off akan mematikan LED dan mengirimkan pesan "LED state set to OFF". Perintah /state akan membaca status LED dan mengirimkan pesan "LED is ON" jika menyala, atau "LED is OFF" jika mati.

Program ini memungkinkan pengguna untuk mengontrol perangkat IoT secara jarak jauh menggunakan Telegram dengan mekanisme sederhana namun efektif. Program ini juga memastikan keamanan dengan membatasi akses kontrol hanya kepada pengguna dengan CHAT_ID yang telah ditentukan. Melalui praktikum ini, kami dapat memahami cara mengintegrasikan ESP32 dengan layanan pesan instan serta menerapkan konsep komunikasi antara perangkat IoT dan pengguna secara real-time.

F. KESIMPULAN

Dari praktikum ini, kami dapat menyimpulkan bahwa penggunaan bot Telegram sebagai media kontrol perangkat IoT, seperti NodeMCU ESP32, merupakan metode yang efektif dan mudah diimplementasikan. Dengan menggunakan pustaka UniversalTelegramBot, perangkat dapat menerima perintah dari pengguna dan meresponsnya secara real-time melalui jaringan internet. Selain itu, program ini telah dirancang dengan mekanisme keamanan menggunakan CHAT_ID, sehingga hanya pengguna yang diotorisasi yang dapat mengendalikan perangkat.

Melalui percobaan ini, kami juga memahami bagaimana cara menghubungkan ESP32 ke jaringan WiFi, mengontrol GPIO untuk menyalakan dan mematikan LED, serta membaca status perangkat melalui komunikasi berbasis internet.